

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1 - CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1 - Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Deepika Shree. S	Reg. No.:	192521264
Department:	B.Tech IT	Date:	

ANNEXURE A
Constraint-Based Problem Solving — Answers

Question 1

Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

Problem Understanding:

The system must service several sensors interrupting at high frequency (every 1 ms, i.e. up to 1000 events/second per sensor) while keeping CPU utilization under a strict 15% budget. This demands an interrupt-driven design with minimal per-event overhead and a carefully chosen I/O addressing technique.

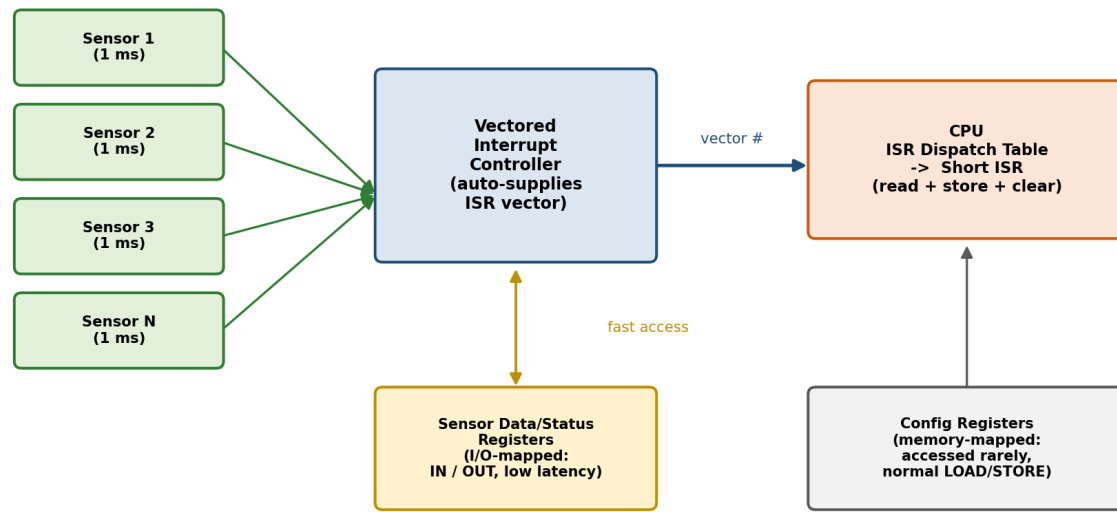
Constraint Analysis:

- CPU < 15% rules out polling; the CPU can only be involved for the brief duration of servicing each interrupt.
- 1 ms interrupt period per sensor, multiplied across several sensors, means the dispatch and ISR must be extremely short to stay within budget.
- Vectored interrupt handling is mandated, so the hardware itself must supply the ISR address - no software polling of interrupt sources is allowed.
- The memory-mapped vs I/O-mapped decision must be driven by latency: sensor registers are accessed very frequently in small amounts, so the fastest, most predictable access path is required.

Solution Development (Constructed Architecture):

Each sensor is wired to a Vectored Interrupt Controller (VIC) with its own vector, so on interrupt the CPU jumps directly to the correct short ISR (read data, store to buffer, clear interrupt, return) without any source-polling overhead. Sensor data/status registers are placed in the I/O-mapped address space, accessed with dedicated IN/OUT instructions, because these give lower and more predictable per-access latency than memory-mapped LOAD/STORE for small, high-frequency register reads. Rarely accessed configuration registers are kept memory-mapped since latency is not critical there and it allows use of the full instruction set. A timing budget check confirms feasibility: if each ISR takes about 1-2 microseconds, then even with 10 sensors interrupting at 1 kHz, CPU busy time is roughly 2% - well inside the 15% ceiling.

Q1: Vectored Interrupt Architecture (CPU < 15%, 1 ms sensor interrupts)



CPU busy time per ISR $\sim 1\text{-}2\ \mu\text{s}$ $\rightarrow$ even 10 sensors @ 1 kHz $\Rightarrow \sim 2\%$ utilization $\ll 15\%$ budget

Application of Concepts:

Vectored interrupts, ISR design and minimization, I/O-mapped vs memory-mapped addressing trade-off, and CPU utilization/timing-budget analysis.

Justification:

The quantitative timing-budget calculation ($\approx 2\%$ at 10 sensors) directly proves the $< 15\%$ constraint is satisfied with margin. Vectoring removes the cost of polling to find the interrupt source, and I/O-mapped access minimizes the per-register latency for the frequently-touched sensor registers, together satisfying both the utilization and latency constraints simultaneously.

Question 2

Design an I/O subsystem under these constraints:

- High-speed storage requires throughput $> 3\ \text{GB/s}$
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

Problem Understanding:

A storage subsystem is required that sustains more than 3 GB/s of throughput using the NVMe protocol over a PCIe link, while ensuring that the CPU never directly moves the bulk data itself.

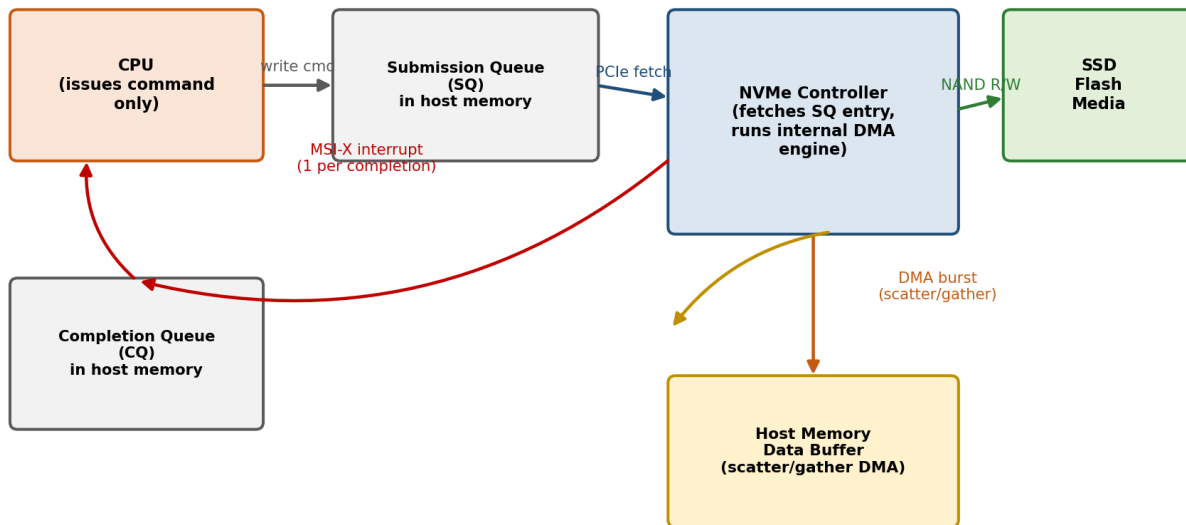
Constraint Analysis:

- $> 3\ \text{GB/s}$ throughput rules out legacy AHCI/SATA; a wide, high-bandwidth PCIe link with a queue-based protocol (NVMe) is required.
- NVMe over PCIe is explicitly mandated - this brings multiple deep parallel submission/completion queues instead of a single command queue.
- "CPU should not handle bulk transfer" means the actual data movement between SSD and host memory must be done by a hardware DMA engine, not by CPU-driven reads/writes.
- DMA and interrupt notification must be integrated so the CPU is informed of completion without needing to poll.

Solution Development (Constructed Architecture):

The CPU only writes a command descriptor into a per-core NVMe Submission Queue (SQ) in host memory. The NVMe controller fetches this command over PCIe and, using its own internal scatter/gather DMA engine, transfers data directly between the SSD flash media and host memory buffers - the CPU's registers are never involved in the data path. A single PCIe Gen4 x4 link provides roughly 7.9 GB/s of raw bandwidth, comfortably exceeding the 3 GB/s requirement, and multiple queues allow this to scale further with parallel workloads. On completion, the controller writes a small completion entry into the Completion Queue (CQ) and raises one MSI-X interrupt per completed command; the CPU's ISR only processes this small metadata entry, never the bulk payload.

Q2: NVMe-over-PCIe I/O Subsystem (>3 GB/s, CPU excluded from bulk transfer)



PCIe Gen4 x4 ~7.9 GB/s per NVMe controller -> satisfies >3 GB/s while CPU only handles small completion entries

Application of Concepts:

NVMe multi-queue command model, PCIe bandwidth scaling, scatter/gather DMA, and MSI-X interrupt-per-completion notification.

Justification:

Because the NVMe controller's DMA engine - not the CPU - performs every byte transfer, the "no bulk transfer by CPU" constraint is met by construction. The PCIe Gen4 x4 link bandwidth (~7.9 GB/s) exceeds the 3 GB/s requirement with headroom, and the completion-queue-plus-single-interrupt model confirms DMA and interrupt notification are integrated exactly as required.

Question 3

Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

Problem Understanding:

Eight devices across two different bus technologies (6 USB, 2 Thunderbolt) must share a common system interconnect fairly, without any device being permanently starved of bus access, while separating time-critical device events from background housekeeping work.

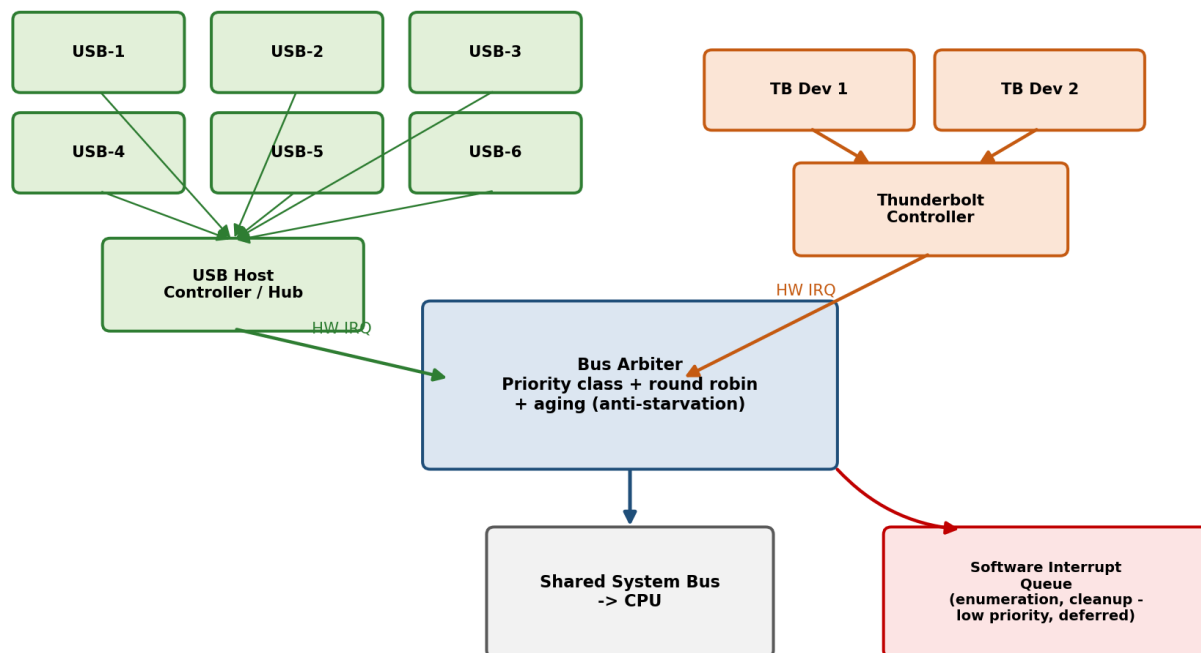
Constraint Analysis:

- Two device classes with very different bandwidth needs (USB peripherals vs high-speed Thunderbolt) must share one interconnect, so a naive single first-come-first-served scheme risks unfairness.
- "Avoid starvation" means a strict fixed-priority scheme (e.g., Thunderbolt always wins) is unacceptable, since sustained Thunderbolt traffic could indefinitely delay USB devices.
- The requirement to use both hardware and software interrupts implies real-time device events and non-urgent background tasks (e.g., enumeration) must be handled through separate paths.
- "Fair bus arbitration" requires that, within a class, no single device dominates either.

Solution Development (Constructed Architecture):

A two-level arbiter is used: Level 1 assigns a base priority class (Thunderbolt higher, due to its bandwidth/latency needs; USB lower), and Level 2 applies round-robin scheduling within each class so the six USB devices take turns fairly among themselves, as do the two Thunderbolt devices. An aging mechanism raises the effective priority of any request that has waited beyond a threshold time, guaranteeing that even lower-class USB requests are eventually serviced no matter how busy the Thunderbolt devices are - this directly prevents starvation. Real-time device events (data ready, error, hot-plug) raise hardware interrupts that go straight to the arbiter/CPU; non-urgent housekeeping tasks such as USB re-enumeration or Thunderbolt security-level negotiation are deferred to a low-priority software interrupt queue so they never block time-critical hardware transfers.

Q3: Fair Multi-Device Bus Arbitration (6 USB + 2 Thunderbolt, starvation-free)



Application of Concepts:

Priority-class + round-robin (weighted fair) bus arbitration, aging for starvation prevention, and separation of hardware vs software interrupts.

Justification:

Round-robin within each class distributes bus time evenly among same-class devices (fairness), while the aging mechanism bounds the maximum wait time for any device regardless of class, which formally eliminates starvation. Routing non-urgent tasks through software interrupts keeps the hardware interrupt path fast and dedicated to real device events, satisfying the dual hardware/software interrupt requirement.

Question 4

Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

Problem Understanding:

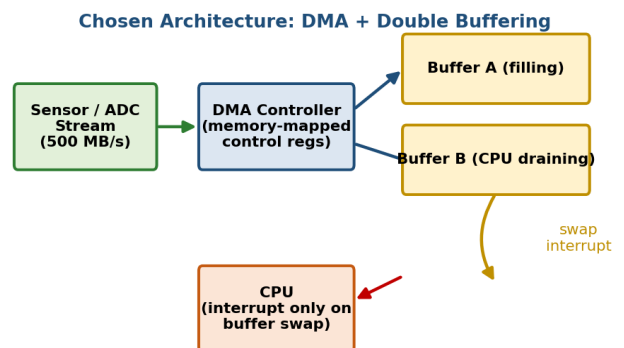
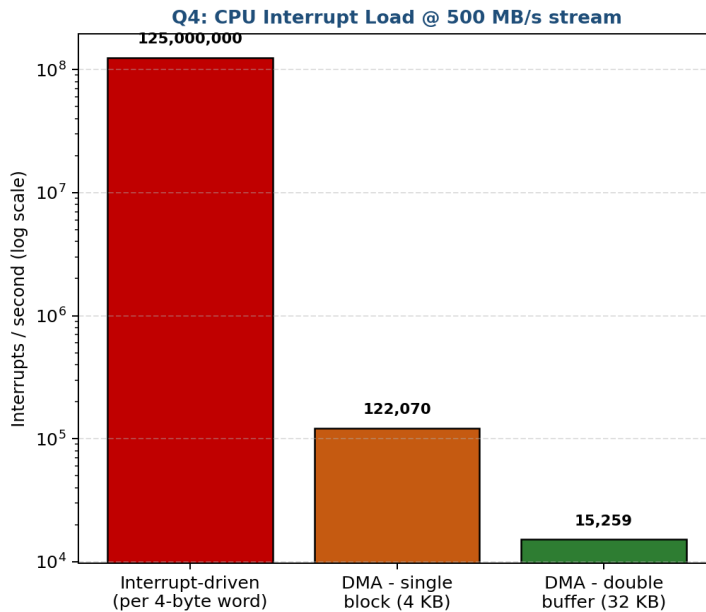
A continuous 500 MB/s acquisition stream (e.g., from an ADC array) must be captured with minimal CPU involvement, requiring an explicit, justified choice between interrupt-driven I/O and DMA, alongside memory-mapped control registers.

Constraint Analysis:

- 500 MB/s is an extremely high sustained rate; the chosen transfer method's per-event CPU overhead must be evaluated quantitatively against this rate.
- "Minimal CPU intervention" means the transfer mechanism itself, not just configuration, must avoid frequent CPU involvement.
- The task explicitly requires comparing DMA vs interrupt-driven I/O, so both must be analyzed rather than assumed.
- Device control/status/configuration registers must use memory-mapped I/O so the CPU can set up and monitor the DAQ hardware with ordinary LOAD/STORE instructions.

Solution Development (Constructed Architecture):

Comparing the two approaches: if the CPU were interrupted per 4-byte data word (interrupt-driven I/O), 500 MB/s implies about 125 million interrupts per second - each costing microseconds for context save/restore - which would completely saturate the CPU and is infeasible. Using DMA with 4 KB block transfers, only about 122,000 interrupts/second are needed; extending this with double (ping-pong) buffering using 32 KB buffers, the CPU is interrupted only when a buffer is completely filled (about 15,000 times/second), and reads/drains the other buffer while the DMA engine fills the next one. The selected architecture therefore uses a DMA engine streaming data continuously into alternating buffers, with the CPU notified only on buffer-swap events; all DAQ control/status/configuration registers are exposed as memory-mapped I/O for straightforward setup and monitoring by the CPU.



Application of Concepts:

Quantitative comparison of DMA vs interrupt-driven I/O overhead, burst/block DMA transfer, double (ping-pong) buffering, and memory-mapped device registers.

Justification:

The interrupt-rate calculation (125,000,000 vs 15,259 interrupts/sec) numerically proves interrupt-driven I/O cannot meet "minimal CPU intervention" at 500 MB/s, while DMA with double buffering reduces CPU involvement by roughly four orders of magnitude, directly satisfying the constraint. Memory-mapped registers are appropriate here since they are accessed only for setup/monitoring, not for the continuous data path.

Question 5

Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Problem Understanding:

An interrupt handling mechanism is required that allows a higher-priority interrupt to preempt (nest within) a lower-priority one currently being serviced, while guaranteeing that no interrupt takes longer than 10 microseconds to begin execution, using vectored dispatch for critical devices and software interrupts for background work.

Constraint Analysis:

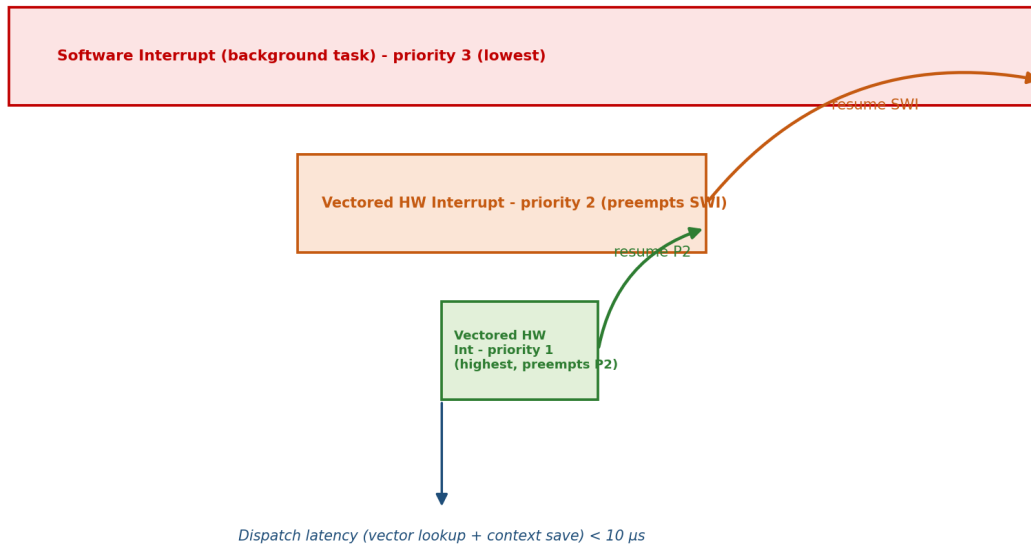
- "Nested interrupts" requires that ISRs be preemptible by higher-priority interrupts, meaning context must be safely saved at each nesting level (stack-based).
- A hard 10 μ s latency bound means the dispatch mechanism (vector lookup + context save) must be extremely fast and bounded in the worst case, including under nesting.
- Vectored interrupts must be used specifically for high-priority devices, avoiding any software search for the correct ISR.
- Background, non-time-critical work must run as software interrupts so it can never block or delay hardware interrupt latency.

Solution Development (Constructed Architecture):

Interrupts are assigned numeric priority levels (0 = highest). High-priority devices use hardware vectored interrupts: on request, the vector table is indexed directly (a single fast lookup) and only the minimal context (PC and flags) is pushed before jumping to the ISR - keeping dispatch latency in the sub-microsecond range, well inside the 10 μ s bound. During ISR execution, only interrupts of equal or lower priority are masked; a higher-priority interrupt is still accepted, causing the CPU to push another context frame on the stack (nesting) and service the new interrupt first, then pop back to resume the interrupted lower-priority ISR once done. Non-time-critical background work (logging, cleanup, deferred processing) is implemented as software interrupts placed in a low-priority queue that the CPU only services when no hardware interrupt is pending, so it can never add to the hardware interrupt's worst-case latency.

Q5: Nested Interrupt Timeline (max latency < 10 μ s)

Stack grows downward with each nested interrupt; only equal/lower-priority interrupts are masked at each level



Application of Concepts:

Priority-based nested interrupt handling, stack-based context nesting, vectored dispatch for deterministic latency, and hardware vs software interrupt separation.

Justification:

Because vector lookup and minimal context save are small, fixed-cost hardware operations (independent of nesting depth for the interrupt currently being accepted), the worst-case dispatch latency stays well under the 10 μ s bound even with nesting. Masking only equal/lower priority levels (not higher) is what enables correct nesting, and offloading background work to software interrupts guarantees it never appears in the hardware interrupt's critical path.

Code of Conduct:

I **Deepika Shree. S (192521264)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____